UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

MSC. DATA SCIENCE

NAME: ISAAC FRIMPONG ASANTE

STUDENT ID: 22424646

COURSE: DSCD 614 - Reinforcement Learning

Assignment 1

Frozen Lake from First Principles Using Q-Learning

JUNE 19th, 2026

Introduction

Reinforcement Learning trains an agent to act through trial and error. The agent reads a state, picks an action, and receives a reward. Over many episodes the agent learns a policy from each state to the best action. This project solves the 8x8 Frozen Lake grid-world with tabular Q-Learning. The environment, the agent, the training loop, and the evaluation run in plain Python. The code uses no Gymnasium, Gym, Stable Baselines, or RLlib.

The agent starts at the top-left cell and walks to the goal at the bottom-right cell. Frozen cells are safe. Holes end the episode with no reward. The goal ends the episode with a reward of one. The agent learns a path around every hole.

Environment Design

The environment represents each state as one integer index, computed as row times eight plus column. The grid holds 64 states numbered 0 to 63. Four actions drive the agent: 0 Left, 1 Down, 2 Right, 3 Up. The environment enforces grid boundaries, so an action into a wall keeps the agent in place. The reward structure stays sparse: one at the goal, zero at a hole, zero on a frozen cell. The FrozenLakeEnv class exposes reset, step, render, get_state, and is_terminal.

Q-Learning Algorithm

Q-Learning learns a table of action values. Each entry Q(s, a) estimates the expected return of action a in state s followed by greedy play. The agent starts the table at zero and refines each entry from experience. The agent implements the update rule as required:

```
Q(s, a) <- Q(s, a) + alpha * [ r + gamma * max Q(s', a') - Q(s, a) ]
```

Alpha is the learning rate and controls update speed. Gamma is the discount factor and weights future reward. The bracket holds the temporal-difference error, and the agent shifts each estimate toward the observed target. The agent explores with an epsilon-greedy rule. With probability epsilon the agent picks a random action, otherwise the highest-value action. Epsilon starts at 1.0 and decays toward 0.01, so early episodes explore and later episodes exploit.

Training Methodology

Each episode resets the agent to the start. The agent acts, reads the reward and next state, and updates the Q-table on every step. Epsilon decays after each episode. The training loop records episode rewards, success flags, and epsilon over time. The hyperparameters appear below.

Parameter	Value	Parameter	Value
Episodes	20000	Epsilon start	1.00
Max steps	200	Epsilon min	0.01
Learning rate alpha	0.10	Epsilon decay	0.9995
Discount gamma	0.99	Map	8x8 Standard

Experimental Results

The greedy evaluation over 100 episodes on the deterministic map reaches 100 percent success, an average reward of 1.0, and zero failures. Across training the success rate rises above 99 percent once epsilon settles near its minimum.

Metric	Result
Evaluation episodes	100
Success rate	100 percent
Average reward	1.0
Failures	0
Successful runs	100

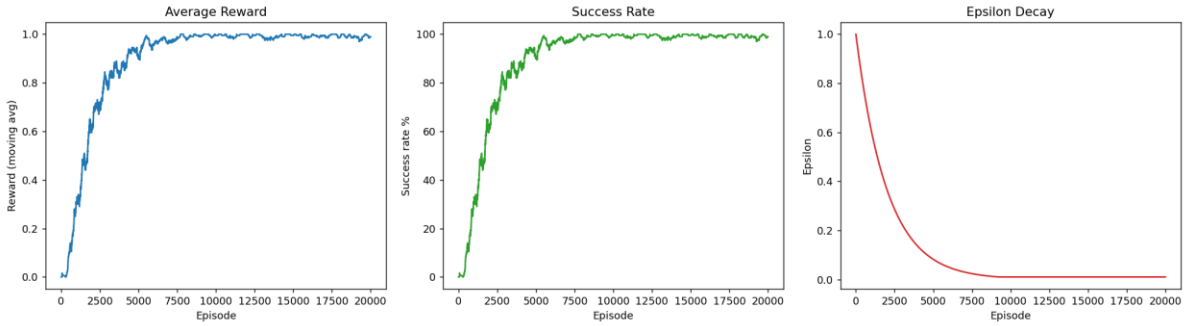

Figure 1. Bonus Option B. Reward, success rate, and epsilon decay across 20000 training episodes. Reward and success rise as epsilon falls.

Learned Policy

The policy comes from reading the best action per non-terminal state from the Q-table. The arrows steer the agent right along the top row, down the right edge, and into the goal, around every hole.

```
↓ → ↓ ↓ ↓ ↓ ↓ ↓
→ → → → → → ↓ ↓
↑ ↑ ↑ H → → → ↓
↑ ↑ ↑ H ↑ → → ↓
↑ ↑ ↑ H → → → ↓
↑ H H → ↑ ↑ H ↓
↑ H ← → H ↑ H ↓
→ ↓ ← H ↓ ↑ ↓ G
```

Challenges Encountered

The sparse reward slowed early learning, since the agent saw a signal only at the goal. A high starting epsilon with slow decay fixed this by forcing wide exploration before exploitation. The decay rate needed care. A fast decay cut exploration too early and left gaps in the Q-table. A slow decay spent episodes on random actions. The chosen decay of 0.9995 balanced both and drove the success rate above 99 percent. Plotting reward, success rate, and epsilon together made these effects clear and guided the tuning.

Conclusion

This project delivers a working Frozen Lake solver from first principles. The tabular Q-Learning agent learns a stable optimal path and scores 100 percent on the deterministic map. The bonus task, Option B, visualizes training performance with graphs of reward, success rate, and epsilon decay. The results confirm that the agent learns a reliable policy once exploration ends.

Appendix. Links

- GitHub Repository: https://github.com/Adomfrimpong/RL-Learning.git
- Report: https://github.com/Adomfrimpong/RL-Learning/blob/main/Report.pdf